Restore: the exact proof command

Verified during the project review, in a separate disposable copy.

1. Open a terminal in the project folder

Use the folder containing demo.py, verify_rollback.py and requirements.txt. Replace the example path below with your own extracted project path.

```
cd "YOUR_PROJECT_FOLDER"
```

2. Run the rollback demonstration

```
python verify_rollback.py
```

This is the actual end-to-end restore command verified during this review. It creates a clean baseline, snapshots the original files, runs a controlled attack, restores the files and compares SHA-256 hashes.

3. Look for the proof, not just a success message

```
restored 30 file(s), verified_identical=True
Proof: 30/30 files byte-for-byte identical
all_recovered=True
```

Recorded result: all 30 originals matched their pre-attack hashes. Counts may differ in another run; the equality checks are what matter.

Important: this resets demo data

Stop the collector and other demo processes first. The script resets the project database and sandbox, creates its own test files and takes snapshots BEFORE the attack. Run it only in a disposable project copy. Never put personal files in the sandbox.

This is a controlled recovery proof, not a command that decrypts arbitrary ransomware-damaged files. Recovery requires a clean saved snapshot. The script prints "quarantined", but actually deletes attacker-added paths after restoring originals.

Source checked: verify_rollback.py, rr/respond/shadow.py. Recorded verification: rollback.log from this review.

Main demo and manual commands

Run from the project folder. These commands are source-checked; they are not all claimed as freshly executed during this review.

Fastest route: the complete presentation demo

```
python demo.py
```

Learns normal activity, trains the model, simulates fast / stealth / header-only attacks, then opens the dashboard. Resets the database and sandbox. It does not run the rollback proof from page 1.

Manual route: use separate terminals

Prepare first, with the collector stopped:

```
python simulator.py --reset --duration 1
```

This resets dummy files, then runs one second of benign activity. The --reset flag alone also runs the default benign mode; it is not reset-only.

Terminal 1 - keep the watcher running:

```
python collector.py
```

Terminal 2 - collect normal activity, then train:

```
python simulator.py --mode benign --duration 60
python model.py train
```

Terminal 2 - run a controlled attack:

```
python simulator.py --mode attack
```

Terminal 3 - open the dashboard:

```
python -m streamlit run app.py
```

Open http://localhost:8501 if the browser does not open automatically. Use either the scripted route or the manual route, not both simultaneously. Ctrl+C stops a foreground process; stop each terminal separately.

Sources checked: README.md, demo.py, simulator.py, collector.py, model.py, app.py.

Other major commands

Useful for testing, comparing attack styles and reproducing experiments.

Attack variants - collector must be running

```
python simulator.py --mode stealth --duration 25
```

Slow full-file rewrites: challenges simple rate thresholds. Uses disposable sandbox files only.

```
python simulator.py --mode partial --duration 25
```

Legacy partial mode changes the first 32 bytes. This is header-only corruption, not the separate v2 15% encryption-fraction setting.

```
python simulator.py --mode bulk
```

Legitimate bulk-copy workload: useful for showing that unusual speed alone is not enough. Reset between independent trials with the collector paused.

Tests - actually executed during this review

```
python -m pytest -q --disable-warnings
```

Recorded result: 73 passed, 2 warnings. This confirms the test suite passed in the review environment; it is not a production-readiness guarantee.

Re-run experiments - optional, changes demo data

```
python experiment.py --quick
python experiment.py
```

Choose one: --quick uses shorter sessions; the second command runs the longer experiment. Both reset the database / sandbox and regenerate results. Save existing evidence first. Do not run beside a live demo.

Setup on a new machine - only if needed

```
python -m pip install -r requirements.txt
```

Use the same Python environment for setup and execution. On Windows, if python is unavailable, use py consistently instead. No real malware is required for these demonstrations.

Sources checked: simulator.py CLI, experiment.py CLI, requirements.txt; tests.log and rollback.log from the review. The verified restore command is python verify_rollback.py, not an assumed python -m rr.respond.shadow demo.